

National University of Computer and Emerging Sciences



Laboratory Manuals

for

Data Structures Lab

(CL -2001)

Course Instructor	Mr. Waqas Rana
Lab Instructor	Ms. Fatima Ali
Lab TA	Mr. Abdur Rehman
Section	BDS-3B
Semester	Fall 2026

*Department of Computer Science
FAST-NU, Lahore, Pakistan*

Lab Manual 02

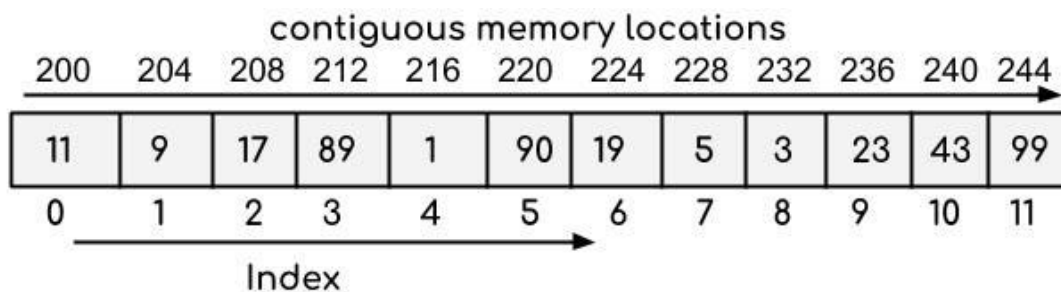
Objectives:

- Understand the concept of a Singly Linked List (SLL).
- Learn how to create a Node class as a nested class inside the LinkedList class.
- Implement basic operations on a linked list:
 - Insert at the start
 - Insert at the end
 - Insert after a given value
 - Search for a value
 - Display all nodes

Singly Linkedlist Implementation in C++

1. Background

In C++ arrays are stored contiguously, so insertions/deletions can be costly.

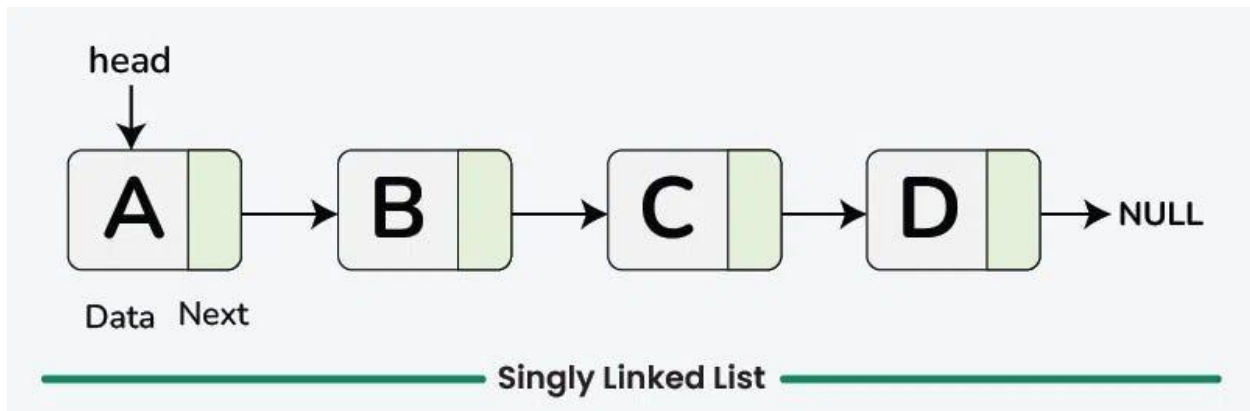


A singly linked list stores nodes at different memory locations and connects them using pointers.

Each node has:

- data (the value)
- next (a pointer to the next node)

We keep track of the first node using a head pointer.



Without a Node class, a linked list cannot exist. But where should we place this Node class?

There are two approaches:

- Global Node Class – defined outside the LinkedList class.
- Nested Node Class – defined inside the LinkedList class.

1. Global Node Class

```
class Node {
public:
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

class LinkedList {
private:
    Node* head;
public:
    LinkedList() : head(nullptr) {}
    // functions...
};
```

- The Node class is defined outside the LinkedList class.
- It is visible to all parts of the program (unless declared private inside a namespace).

Advantages

- a. Reusability – The same Node class can be reused in different data structures (e.g., LinkedList, Stack, Queue).
- b. Simplicity – Easy to teach beginners, as structure is separated from list logic.
- c. Clarity – Students can visualize Node as an independent building block.

Disadvantages

- a. Less Encapsulation – Node is globally accessible, even where it shouldn't be used.
- b. Clutter in Namespace – If many data structures are written, all their Node classes may conflict.
- c. Tight Coupling Risk – Node can be misused by code outside LinkedList.

2. Nested Node Class

```
class LinkedList {
private:
    class Node {
    public:
        int data;
        Node* next;
        Node(int val) : data(val), next(nullptr) {}
    };

    Node* head;

public:
    LinkedList() : head(nullptr) {}
    // functions...
};
```

- The Node class is defined inside the LinkedList class.
- It becomes a private/internal detail of the list.

Advantages

- Encapsulation – Node is hidden inside LinkedList. No external code can directly create or misuse nodes.
- Clean Design – Only LinkedList is exposed to users, making API simpler.

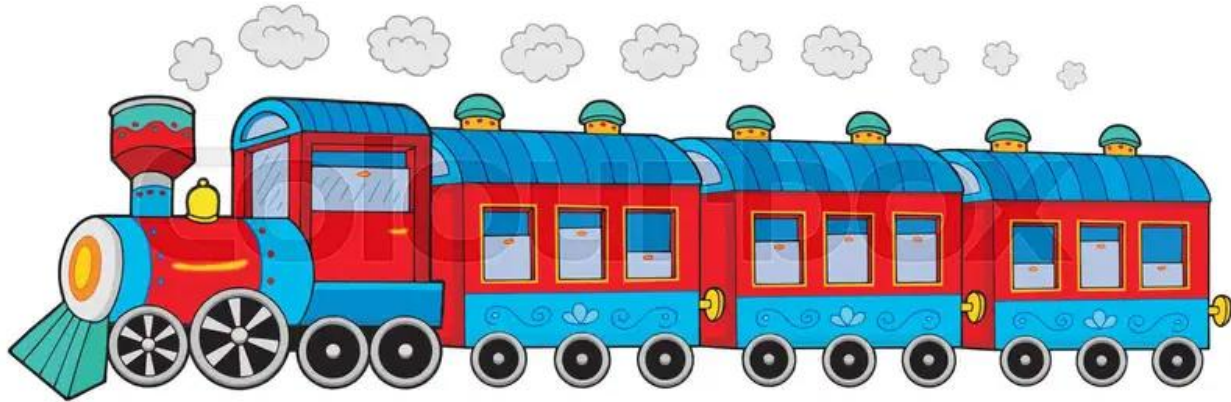
- Maintainability – If Node changes internally, external code is unaffected.

Disadvantages

- Less Reusable – You cannot reuse the Node class in another data structure (e.g., a separate Queue implementation).
- Code Duplication Risk – If multiple data structures (like LinkedList, Stack, Queue) all need nodes, each will have to define its own nested Node.

Best Practices

- Use nested Node class for encapsulation.
- Always manage memory carefully (new and delete).
- Handle edge cases properly.
- Keep functions modular, readable, and testable.
- Use temporary pointers for traversal; never modify head unintentionally.
- Maintain head (and optionally tail) pointers correctly.
- Comment and document code for clarity.



Context

Imagine Thomas' train on Sodor Island. Each compartment is connected one after another, carrying passengers, coal, mail, or toys. Think of this train as a Singly Linked List (SLL): each compartment is a node with its data (like ID and cargo type) and a pointer to the next compartment. The first compartment is the head, and if the train is empty, the head is null.

When Thomas adds a VIP compartment at the front, it's like inserting at the start of a linked list. Adding a normal compartment at the end is like inserting at the end, and inserting after a specific compartment is like inserting in the middle. If a compartment is detached, that's similar to deleting a node, and checking whether a compartment exists is like searching through the list.

So, by imagining a train where each compartment is linked to the next and carries different cargo types including passengers, coal, mail, or toys; you can easily picture how a Singly Linked List works dynamically, with nodes connected one after another, just like Thomas' compartments on the track.

Requirements

1. Create Train Class (SLL) having nested compartment class (Node).
2. Implement a menu driven Program to support following functionalities

```
=== Thomas & Friends: Train Compartment Manager ===
1. Attach Compartment at Start (VIP)
2. Attach Compartment at End (Normal)
3. Attach Compartment After a Specific Compartment
4. Detach Compartment
5. Search Compartment
6. Display Train
7. Count Compartments
9. Swap Two Compartments
0. Exit
Enter your choice:
```

1. Insert at Head

```
Enter your choice: 1
Enter Compartment ID: 1
Enter Cargo Type (Passengers, Coal, Mail, Toys): Toys
Compartment 1 (Toys) attached at start.
```

2. Insert at End

```
Enter your choice: 2
Enter Compartment ID: 1
Enter Cargo Type (Passengers, Coal, Mail, Toys): Mail
Compartment 1 (Mail) attached at end.
```

3. Display Train Composition

```
Enter your choice: 6
Train Composition:
1 (Toys) -> 1 (Mail) -> NULL
```

4. Search a Compartment

```
Enter your choice: 5
Enter Compartment ID to search: 1
Compartment 1 is in the train.
```

5. Insert After a Specific Compartment

```
Enter your choice: 3
Enter Target Compartment ID: 1
Enter New Compartment ID: 3
Enter Cargo Type (Passengers, Coal, Mail, Toys): Passengers
Compartment 3 (Passengers) attached after 1.
```

```
Enter your choice: 3
Enter Target Compartment ID: 4
Enter New Compartment ID: 4
Enter Cargo Type (Passengers, Coal, Mail, Toys): Toys
Target compartment 4 not found.
```

6. Detach a Compartment

```
Enter your choice: 4
Enter Compartment ID to detach: 3
Compartment 3 detached.
```

7. Count Compartments

```
Enter your choice: 6
Train Composition:
2 (Coal) -> 1 (Toys) -> 3 (Mail) -> 4 (Passengers) -> NULL
```

```
Enter your choice: 7
Total Compartments: 4
```

8. Swap Two Compartments

```
Enter your choice: 9
Enter first Compartment ID: 3
Enter second Compartment ID: 2
Compartments 3 and 2 swapped.
```

```
Enter your choice: 6
Train Composition:
4 (Passengers) -> 2 (Coal) -> 1 (Toys) -> 3 (Mail) -> NULL
```

9. Destruct the Train properly

Submission Guidelines

1. submit following the only file strictly following the naming convention
 - a. l261234.cpp